Análisis de Cursos Udemy — Procesamiento y Análisis de Datos

Autor: Melany Quiceno - Natalia Lozano

Dataset: udemy_courses_data.csv

Objetivo: Visualizar hallazgos, construir narrativa basada en datos y entrenar un modelo predictivo.

Etapa 1 — Limpieza y preparación de datos

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.gridspec import GridSpec
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error
import warnings
import re
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
warnings.filterwarnings('ignore')

# Carga
df = pd.read_csv('/content/drive/MyDrive/udemy_courses_data.csv', sep=',')
print('Shape:', df.shape)
df.head(3)
```

```
Shape: (3443, 11)
```

	index	title	description	difficulty	certificate_available	students	price	rating	reviews_count	source
0	datascience/	Data Science A-Z: Hands-On Exercises & ChatGPT...	Learn Data Science step by step through real A...	Все уровни	True	223 757 студента	10890.0	4,6	34751	udemy
1	machine-learning-for-absolute-	Machine Learning for Absolute	Learn the Fundamental Concepts of	Начальный	True	120 723 студента	6090.0	4,5	31061	udemy

Próximos pasos: Generar código con df New interactive sheet

```
# ── Limpieza de columna 'students debido a que hay dato sen ruso' ──────────────────────────
def clean_students(s):
    s = str(s).replace('\xa0', '').replace(' ', '')
    digits = ''.join(c for c in s if c.isdigit())
    return int(digits) if digits else 0

df['students_clean'] = df['students'].apply(clean_students)

# ── Limpieza de columna 'rating' ──────────────────────────────────────────
# Viene con coma decimal rusa, ej: '4,6'
df['rating_clean'] = df['rating'].astype(str).str.replace(',', '.').astype(float)

# ── Mapeo de dificultad al español ────────────────────────────────────────
diff_map = {
    'Все уровни': 'Todos los niveles',
    'Начальный': 'Principiante',
    'Средний': 'Intermedio',
    'Эксперт': 'Experto' # <-- Este es el que aparece en tu archivo
}
df['difficulty_es'] = df['difficulty'].map(diff_map)

# -- Datos repetidos
```

```
df.drop_duplicates(subset=['title'], keep='first', inplace=True)
print(f"Filas después de eliminar duplicados: {len(df)}")  # → 2573

# ── Dataset sin cursos sin reseñas (para modelo) ──────────────────────────
df_model = df[df['rating_clean'] > 0].copy()

print(f'Cursos totales: {len(df)}')
print(f'Cursos con rating válido: {len(df_model)}')
print(f'Valores nulos:\n{df.isnull().sum()}')
df_model[['students_clean','price','rating_clean','reviews_count']].describe().round(2)

# Convertir la dificultad a números (0, 1, 2, 3) para que la regresión pueda entenderla
df_model['diff_code'] = df_model['difficulty_es'].astype('category').cat.codes
```

```
Filas después de eliminar duplicados: 2573
Cursos totales: 2573
Cursos con rating válido: 2446
Valores nulos:
index                     0
title                     0
description               1
difficulty                0
certificate_available     0
students                  0
price                     0
rating                    0
reviews_count             0
source                    0
course                    0
students_clean            0
rating_clean              0
difficulty_es            72
dtype: int64
```

Etapa 2 — Visualizaciones y Storytelling

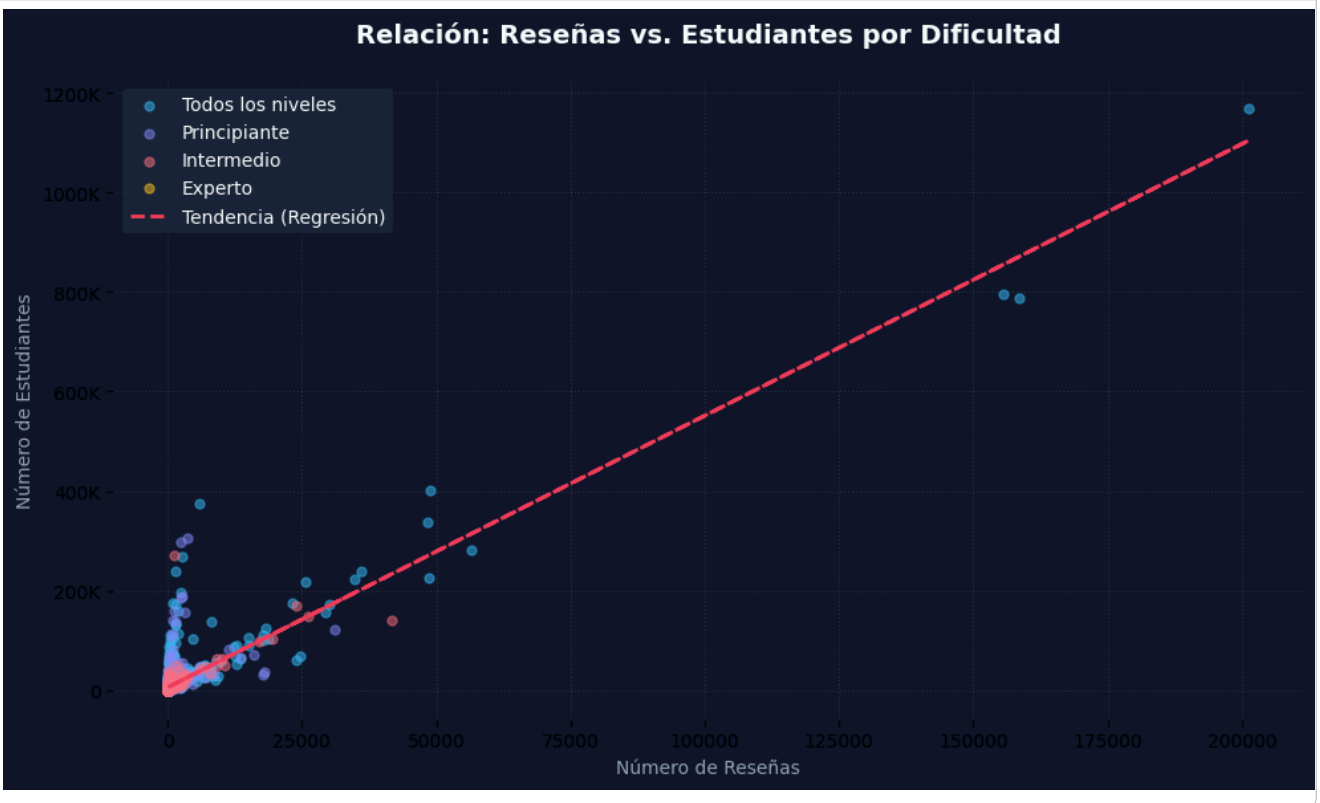
Historia: ¿Qué nos dice el catálogo de Udemy sobre el mercado de aprendizaje en línea?

Con **3 443 cursos** en la categoría *Procesamiento y Análisis de Datos*, Udemy ofrece una ventana única para entender cómo se distribuye la demanda, los precios y la calidad percibida en la educación tecnológica.

```
# Paleta de colores completa para que nada falle
BG, CARD, TEXT, SUBTEXT = '#0F172A', '#1E293B', '#F8FAFC', '#94A3B8'
ACC, ACC2, WARN = '#38BDF8', '#818CF8', '#F43F5E'

DIFF_COLORS = {
    'Todos los niveles': ACC,
    'Principiante': ACC2,
    'Intermedio': '#FB7185',
    'Experto': '#FBBF24'
}

# ── PREPARACIÓN DE DATOS PARA EL GRÁFICO ──────────────────────────────────
X_plot = df_model[['reviews_count']] # Variable independiente
y_plot = df_model['students_clean']  # Variable dependiente

# Entrenar un modelo rápido solo para la línea de tendencia de la gráfica
reg_line = LinearRegression()
reg_line.fit(X_plot, y_plot)
y_pred_line = reg_line.predict(X_plot)
```

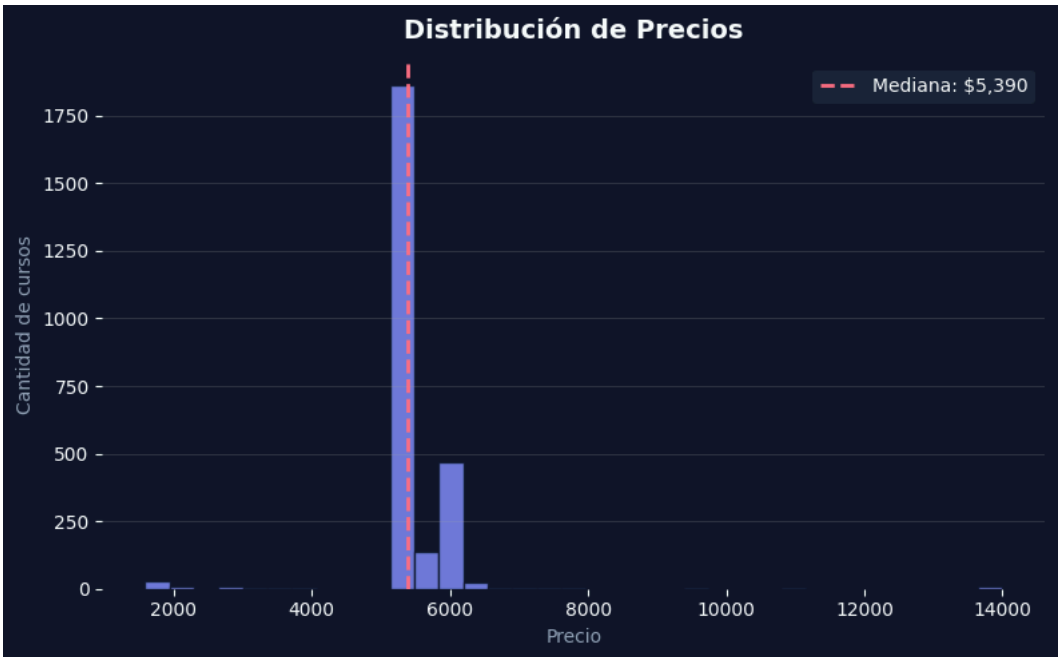
```
fig, ax = plt.subplots(figsize=(10, 6), facecolor=BG)
ax.set_facecolor(BG)

# Dibujar puntos por cada categoría de dificultad
for diff, color in DIFF_COLORS.items():
    mask = df_model['difficulty_es'] == diff
    ax.scatter(df_model[mask]['reviews_count'],
               df_model[mask]['students_clean'],
               color=color, alpha=0.5, s=25, label=diff)

# Dibujar línea de regresión
ax.plot(X_plot, y_pred_line, color=WARN, linewidth=2, linestyle='--', label='Tendencia (Regresión)')
```

```
# Títulos y etiquetas en español
ax.set_title('Relación: Reseñas vs. Estudiantes por Dificultad', color=TEXT, fontsize=14, fontweight='bold', pad=20)
ax.set_xlabel('Número de Reseñas', color=SUBTEXT)
ax.set_ylabel('Número de Estudiantes', color=SUBTEXT)

ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{int(x/1000)}K' if x >= 1000 else int(x)))

legend = ax.legend(facecolor=CARD, edgecolor='none', labelcolor=TEXT, loc='upper left')
ax.grid(True, linestyle=':', alpha=0.2, color=SUBTEXT)

for spine in ax.spines.values():
    spine.set_visible(False)

plt.tight_layout()
plt.show()
```

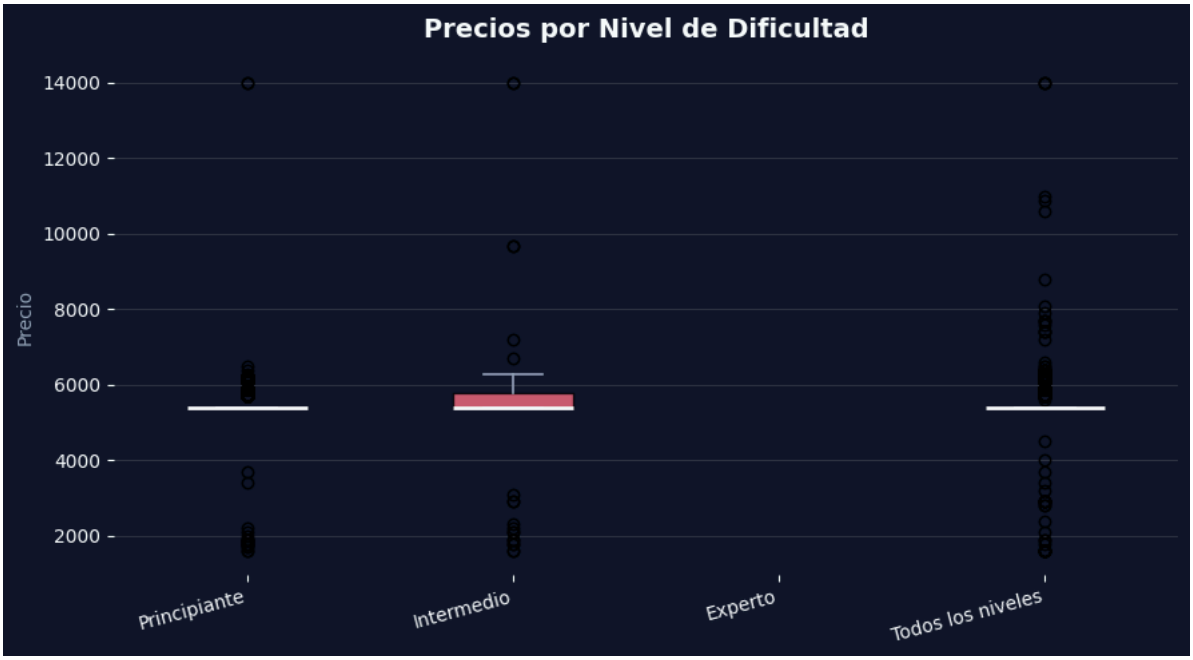



```
# ── Gráfico 2: Distribución de precios (CORREGIDO) ──────────────────────────
# Definimos ACC3 para evitar el NameError
ACC3 = '#FB7185' # Un tono rosado/coral que combina con tu paleta

fig, ax = plt.subplots(figsize=(8, 5), facecolor=BG)
ax.set_facecolor(BG) # Asegura el fondo oscuro

ax.hist(df['price'], bins=35, color=ACC2, edgecolor=BG, alpha=0.85)
median_p = df['price'].median()

# Ahora ACC3 ya existe y el código no se detendrá
ax.axvline(median_p, color=ACC3, linestyle='--', linewidth=2,
           label=f'Mediana: ${median_p:,.0f}')

ax.set_title('Distribución de Precios', fontsize=14, color=TEXT,
             fontweight='bold', pad=12)
ax.set_xlabel('Precio', color=SUBTEXT)
ax.set_ylabel('Cantidad de cursos', color=SUBTEXT)

# Ajuste de colores de los números en los ejes
ax.tick_params(colors=TEXT)

ax.legend(fontsize=10, facecolor=CARD, labelcolor=TEXT, edgecolor='none')
ax.grid(axis='y', alpha=0.2)

# Quitar bordes para estilo moderno
```

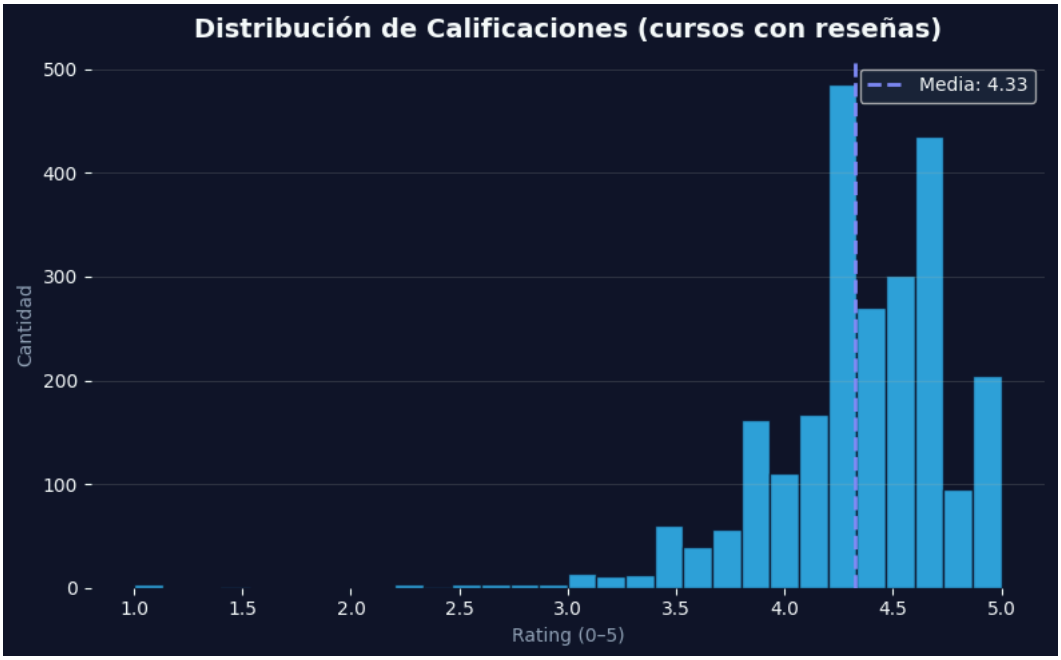
```
for spine in ax.spines.values():
    spine.set_visible(False)

plt.tight_layout()
plt.show()
```



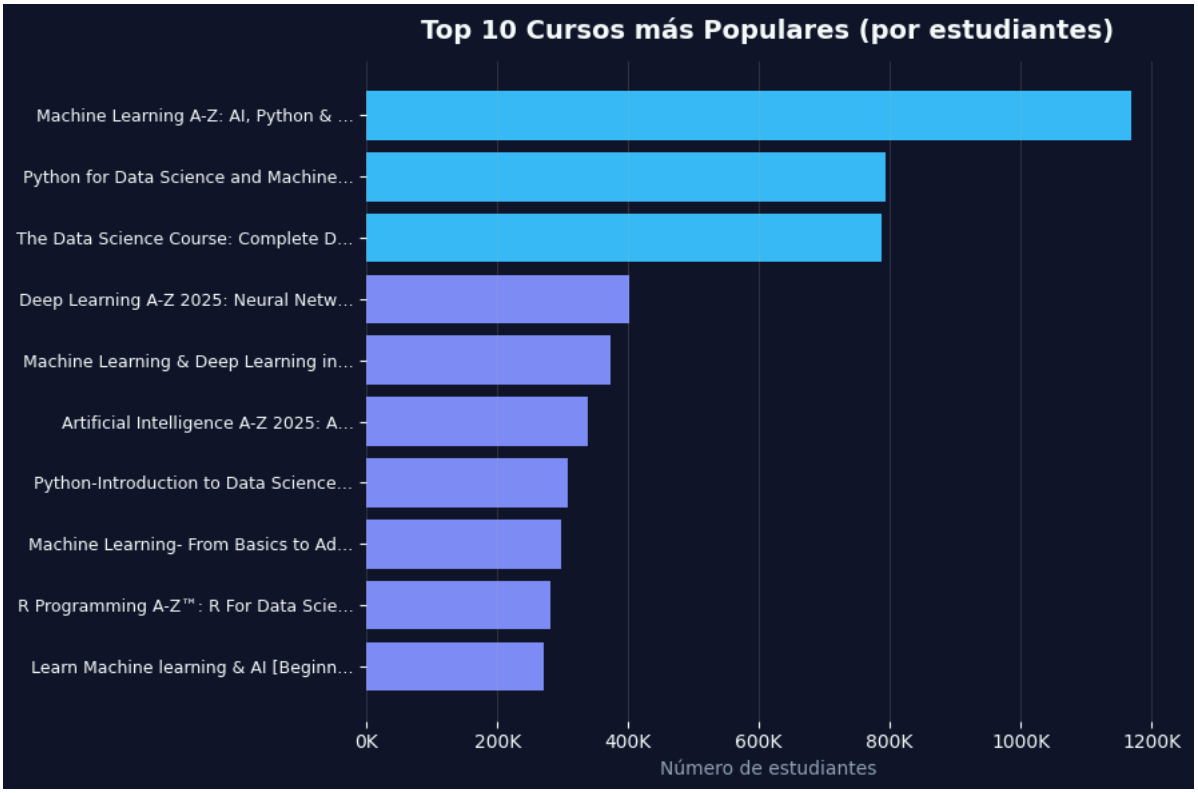

```
# ── Gráfico 3: Boxplot precio por dificultad (CORREGIDO) ──────────────────
fig, ax = plt.subplots(figsize=(9, 5), facecolor=BG)
ax.set_facecolor(BG)

# Ajustamos el orden para que coincida con tu nuevo mapeo (Experto)
order = ['Principiante', 'Intermedio', 'Experto', 'Todos los niveles']

# Usamos 'difficulty_es' que es la columna que ya tiene los nombres en español
data_box = [df[df['difficulty_es'] == d]['price'].dropna().values for d in order]

bp = ax.boxplot(data_box, patch_artist=True,
                medianprops=dict(color=TEXT, linewidth=2))

# Aplicar colores (usando el diccionario DIFF_COLORS que definimos antes)
for patch, d in zip(bp['boxes'], order):
    patch.set_facecolor(DIFF_COLORS.get(d, ACC)) # .get evita errores si falta un color
    patch.set_alpha(0.8)

for el in ['whiskers', 'caps', 'fliers']:
    for item in bp[el]: item.set(color=SUBTEXT, linewidth=1.2)

ax.set_xticklabels(order, rotation=15, ha='right', color=TEXT)
ax.set_title('Precios por Nivel de Dificultad', fontsize=14, color=TEXT,
             fontweight='bold', pad=12)
ax.set_ylabel('Precio', color=SUBTEXT)
ax.tick_params(colors=TEXT) # Asegura que los números de los ejes se vean
ax.grid(axis='y', alpha=0.2)

# Eliminar bordes
for spine in ax.spines.values(): spine.set_visible(False)

plt.tight_layout()
plt.show()
```




```
# ── Gráfico 4: Distribución de calificaciones (CORREGIDO) ──────────────────
fig, ax = plt.subplots(figsize=(8, 5), facecolor=BG)
ax.set_facecolor(BG)

# Usamos la columna limpia que creamos: 'rating_clean'
df_rated = df[df['rating_clean'] > 0]
mean_r = df_rated['rating_clean'].mean()

# Usamos ACC (celeste) y ACC2 (morado) para los colores
ax.hist(df_rated['rating_clean'], bins=30, color=ACC, edgecolor=BG, alpha=0.85)
ax.axvline(mean_r, color=ACC2, linestyle='--', linewidth=2,
           label=f'Media: {mean_r:.2f}')

ax.set_title('Distribución de Calificaciones (cursos con reseñas)', fontsize=14,
             color=TEXT, fontweight='bold', pad=12)
ax.set_xlabel('Rating (0–5)', color=SUBTEXT)
ax.set_ylabel('Cantidad', color=SUBTEXT)
ax.tick_params(colors=TEXT)
ax.legend(fontsize=10, facecolor=CARD, labelcolor=TEXT)
ax.grid(axis='y', alpha=0.2)

# Eliminar bordes
for spine in ax.spines.values(): spine.set_visible(False)

plt.tight_layout()
plt.show()
```




```
# ── Gráfico 5: Top 10 cursos por estudiantes (CORREGIDO) ──────────────────
fig, ax = plt.subplots(figsize=(9, 6), facecolor=BG)
ax.set_facecolor(BG)

# Usamos 'students_clean' que es la columna numérica
top10 = df.nlargest(10,'students_clean')[['title','students_clean']].copy()
top10['title_short'] = top10['title'].str[:35] + '…'

# Usamos los colores ACC y ACC2 que definimos al principio
colors_top = [ACC if i < 3 else ACC2 for i in range(len(top10))]

ax.barh(range(len(top10)), top10['students_clean'], color=colors_top, edgecolor='none')
ax.set_yticks(range(len(top10)))
ax.set_yticklabels(top10['title_short'], fontsize=9, color=TEXT)
ax.invert_yaxis()

ax.set_title('Top 10 Cursos más Populares (por estudiantes)', fontsize=14,
             color=TEXT, fontweight='bold', pad=12)
ax.set_xlabel('Número de estudiantes', color=SUBTEXT)
ax.tick_params(colors=TEXT)
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x,_: f'{x/1e3:.0f}K'))
ax.grid(axis='x', alpha=0.2)

for spine in ax.spines.values(): spine.set_visible(False)
plt.tight_layout()
plt.show()
```



Etapa 3 — Modelo Predictivo: Regresión Lineal

Pregunta: ¿Podemos predecir el número de estudiantes de un curso a partir de sus reseñas?

Hipótesis: A más reseñas, más estudiantes. La relación debería ser positiva y fuerte.

```
# ── Preparación de datos para el modelo ──────────────────────────────
# Usamos df_model que ya tiene filtrados los ratings > 0
X = df_model[['reviews_count']].values
y = df_model['students_clean'].values

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ── Entrenamiento ────────────────────────────────────────────────────
model = LinearRegression()
model.fit(X_train, y_train)

# ── Evaluación ───────────────────────────────────────────────────────
y_pred = model.predict(X_test)
r2  = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

print('=' * 50)
print('RESULTADOS DEL MODELO')
print('=' * 50)
print(f'Coeficiente (pendiente): {model.coef_[0]:.2f}')
print(f'  → Por cada reseña adicional, se esperan ~{model.coef_[0]:.1f} estudiantes más')
print(f'Intercepto:              {model.intercept_:.0f}')
print(f'R² Score:                {r2:.4f}  ({r2*100:.1f}% de varianza explicada)')
print(f'MAE:                     {mae:,.0f} estudiantes')
print('=' * 50)
```

```
==================================================
RESULTADOS DEL MODELO
==================================================
Coeficiente (pendiente): 5.44
  → Por cada reseña adicional, se esperan ~5.4 estudiantes más
Intercepto:              5397
R² Score:                0.2939  (29.4% de varianza explicada)
MAE:                     9,688 estudiantes
==================================================
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6), facecolor=BG)

# Bloque Izquierdo: Reseñas vs Estudiantes
ax = axes[0]
ax.set_facecolor(BG)

# Filtro para que la gráfica no se vea "aplastada" por valores atípicos extremos
mask = (df_model['reviews_count'] < 30000) & (df_model['students_clean'] < 500000)
df_vis = df_model[mask]

# Mapeo de colores corregido
scatter_c = [DIFF_COLORS.get(d, ACC) for d in df_vis['difficulty_es']]

ax.scatter(df_vis['reviews_count'], df_vis['students_clean'],
           c=scatter_c, alpha=0.4, s=15, edgecolors='none')

# Línea de regresión
x_range = np.linspace(0, 30000, 200).reshape(-1, 1)
ax.plot(x_range, model.predict(x_range), color=WARN, linewidth=2.5, label='Regresión lineal')

ax.set_title('Reseñas vs Estudiantes', fontsize=13, color=TEXT, fontweight='bold', pad=12)
ax.set_xlabel('Número de reseñas', color=SUBTEXT)
ax.set_ylabel('Número de estudiantes', color=SUBTEXT)
ax.tick_params(colors=TEXT)

# Leyenda con parches de color
legend_p = [mpatches.Patch(color=color, label=label) for label, color in DIFF_COLORS.items()]
legend_p.append(plt.Line2D([0],[0], color=WARN, linewidth=2, label='Regresión'))
ax.legend(handles=legend_p, fontsize=8, facecolor=CARD, edgecolor='none', labelcolor=TEXT)

# Bloque Derecho: Predichos vs Reales
ax2 = axes[1]
ax2.set_facecolor(BG)
mask2 = y_test < 200000
ax2.scatter(y_test[mask2], y_pred[mask2], color=ACC2, alpha=0.4, s=15)

lim = max(y_test[mask2].max(), y_pred[mask2].max())
ax2.plot([0, lim], [0, lim], color=WARN, linewidth=2, linestyle='--', label='Predicción perfecta')

ax2.set_title('Valores Reales vs Predichos', fontsize=13, color=TEXT, fontweight='bold', pad=12)
ax2.set_xlabel('Estudiantes reales', color=SUBTEXT)
ax2.set_ylabel('Estudiantes predichos', color=SUBTEXT)
ax2.tick_params(colors=TEXT)
ax2.legend(fontsize=9, facecolor=CARD, edgecolor='none', labelcolor=TEXT)

# Formateo de miles (K)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x,_: f'{x/1e3:.0f}K'))
ax2.xaxis.set_major_formatter(plt.FuncFormatter(lambda x,_: f'{x/1e3:.0f}K'))
ax2.yaxis.set_major_formatter(plt.FuncFormatter(lambda x,_: f'{x/1e3:.0f}K'))

plt.tight_layout()
plt.show()
```

```
# ── Ejemplo de predicción ──────────────────────────────────────────────
ejemplos = [100, 500, 1000, 5000]
print('Ejemplos de predicción:')
print('-' * 40)
for rev in ejemplos:
    pred = model.predict([[rev]])[0]
    print(f'  {rev:5,} reseñas → ~{pred:,.0f} estudiantes estimados')
```

```
Ejemplos de predicción:
----------------------------------------
  100 reseñas → ~5,941 estudiantes estimados
  500 reseñas → ~8,116 estudiantes estimados
1,000 reseñas → ~10,835 estudiantes estimados
5,000 reseñas → ~31,900 estudiantes estimados
```

```
# --- TRADUCCIÓN de Ruso ---

df['difficulty'] = df['difficulty'].astype(str).str.strip()
df['course'] = df['course'].astype(str).str.strip()

mapa_temas = {
    'Обработка и анализ данных': 'Procesamiento y Análisis de Datos'
}

#Diccionario para Dificultad
mapa_dificultad = {
    'Все уровни': 'Todos los niveles',
    'Начальный': 'Principiante',
    'Средний': 'Intermedio',
    'Эксперт': 'Experto',
    'Профессиональный': 'Experto'
}

# Aplicar las traducciones
```